

Contents

Kilonova SED pipeline — user guide	1
1. Prerequisites	2
1.1 Software	2
1.2 The COCO_PATH environment variable	2
1.3 Headless plotting	2
1.4 Repository layout	2
2. First complete run (AT2017gfo)	3
2.1 Environment check	3
2.2 Optional reset	3
2.3 Preparation notebooks (if needed)	3
2.4 Production scripts	3
2.5 What each command reads and writes	3
2.6 Expected artifacts checklist	4
3. Input data reference	4
3.1 Photometry stages	4
3.2 Spectroscopy stages	5
3.3 Filters	5
3.4 Supernova info table	6
3.5 Scale groups JSON	6
4. Notebooks — guidance	6
4.1 0.1_Smooth_spectra_KN.ipynb	6
4.2 1_LC_DustCorrection_KN.ipynb	7
4.3 2_LC_modelRising_KN_fullfit_log.ipynb	7
4.4 LCfit_publication_plots.ipynb (optional)	7
4.5 Downstream QA notebooks (7.5_*.ipynb, bolometric_luminosity.ipynb, plotting-final.ipynb)	7
5. Script drivers	7
5.1 3_LCfit_KN_log.py	7
5.2 4_Scale_spectra_KN.py	8
5.3 5_Mangle_spectra_KN_log.py	9
5.4 6_Iterative_GP_mangle_KN.py	9
5.5 clear_sn_run.py	10
6. Iteration plot layout and quality assurance	10
7. pipeline_config.py reference	11
7.1 Object identity and paths	11
7.2 Prescale (step 4)	11
7.3 Mangling (step 5 and remangle in step 6)	12
7.4 Iterative two-dimensional Gaussian Process (step 6)	12
7.5 Plotting style	12
8. Troubleshooting	12
9. Developer documentation and PDF generation	13

Kilonova SED pipeline — user guide

This document is the full instruction manual for **kn-sed-pipeline**: a standalone log-space photometry and spectroscopy pipeline that builds a time-evolving spectral energy distribution (SED) from multi-band light curves and sparse spectra. The default worked example is **AT2017gfo**.

Read this guide when you need to run the pipeline end to end, prepare a new transient, interpret outputs, or change configuration. For a shorter first-pass overview, start with the repository [README](#). For equations and scientific rationale, see the [Pipeline Writeup](#).

1. Prerequisites

1.1 Software

You need:

- **Python 3.10 or newer** (Python 3.12 is tested in development).
- A virtual environment is strongly recommended.
- All packages in [requirements.txt](#). The non-standard dependency that most often needs attention is **george** (Gaussian Process library). Also required: numpy, scipy, pandas, matplotlib, astropy, extinction, synphot, emcee, scikit-learn, and jupyter (for preparation notebooks).

Install with:

```
cd /path/to/kn-sed-pipeline
python -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

1.2 The COCO_PATH environment variable

Every notebook and script resolves paths relative to the repository root. That root is taken from the environment variable `COCO_PATH`, or, if unset, from the parent of the `Codes/` directory when `pipeline_config.py` is imported.

```
export COCO_PATH="/path/to/kn-sed-pipeline"
```

`COCO_PATH` must point at the folder that contains `Codes/`, `Inputs/`, and `Outputs/`. Trailing slashes are normalized by the configuration module.

1.3 Headless plotting

On servers without a display, set:

```
export MPLBACKEND=Agg
```

Production scripts already force the Agg backend where they create figures. Setting it in the shell avoids failures when you import plotting helpers from an interactive session.

1.4 Repository layout

Path	Purpose
<code>Codes/</code>	Preparation notebooks, production <code>.py</code> drivers, and <code>pipeline_config.py</code>
<code>Codes/helper_scripts/</code>	Shared libraries (light-curve fit, prescale, mangling, iterative Gaussian Process)
<code>Codes/helper_scripts/twodim_gp/</code>	Vendored two-dimensional Gaussian Process inference and plots
<code>Codes/helper_scripts/docs/</code>	Developer phase notes (prescale, iteration, data flow, QA)
<code>Codes/helper_scripts/tests/</code>	Unit tests
<code>Inputs/Photometry/</code>	Light-curve stages (<code>1_raw</code> → <code>2_dust-corrected</code> → <code>3_extrapolated</code>)
<code>Inputs/Spectroscopy/</code>	Original, smoothed, and prescaled spectra plus list files
<code>Inputs/Filters/</code>	Filter throughput tables (<code>GeneralFilters/</code> and variants)
<code>Inputs/SNe_Info/</code>	Metadata table (<code>info.dat</code>) used by dust correction
<code>Inputs/2DIM_priors/</code>	Optional surface priors for the two-dimensional Gaussian Process
<code>Outputs/<SN>/</code>	All pipeline products for one supernova or kilonova name
<code>docs/</code>	This user guide, the scientific writeup, and PDF build scripts

2. First complete run (AT2017gfo)

This section walks through a full production run on the shipped AT2017gfo inputs. It assumes notebooks 0.1–2 have already produced smoothed spectra and an extrapolated light curve (as in the repository checkout), or that you are willing to run those notebooks first.

2.1 Environment check

```
export COCO_PATH="/path/to/kn-sed-pipeline"
cd "$COCO_PATH/Codes"
source ../.venv/bin/activate # if you use a venv
python -c "import pipeline_config as p; print(p.COCO_PATH); print(p.SNNAME_DEFAULT)"
```

You should see your repository path and AT2017gfo.

2.2 Optional reset

If a previous run left stale products:

```
python clear_sn_run.py --snname AT2017gfo --yes
```

This empties `Outputs/AT2017gfo/` but keeps the directory. Prescaled spectra under `Inputs/` are **not** deleted unless you also pass `--include-input-intermediates` (requires `--yes`). Use `--dry-run` to list targets without deleting.

2.3 Preparation notebooks (if needed)

If `Inputs/Photometry/3_LCs_extrapolated/AT2017gfo.dat` and `Inputs/Spectroscopy/2_spec_lists_smoothed/AT2017gfo.dat` already exist, you may skip to the scripts. Otherwise open Jupyter from `Codes/` and run:

1. `0.1_Smooth_spectra_KN.ipynb`
2. `1_LC_DustCorrection_KN.ipynb`
3. `2_LC_modelRising_KN_fullfit_log.ipynb`

Confirm that `SN_EXPLOSION_MJD["AT2017gfo"]` in `pipeline_config.py` matches the explosion (merger) time used in notebook 2.

2.4 Production scripts

```
cd "$COCO_PATH/Codes"
```

```
python 3_LCfit_KN_log.py --snname AT2017gfo
```

```
python 4_Scale_spectra_KN.py --snname AT2017gfo --write-groups-only
# Open Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json and confirm
# XSHOOTER UVB/VIS/NIR groups and merge_order before continuing.
python 4_Scale_spectra_KN.py --snname AT2017gfo
```

```
python 5_Mangle_spectra_KN_log.py --snname AT2017gfo --save-epoch-plots
```

```
python 6_Iterative_GP_mangle_KN.py --snname AT2017gfo --max-iters 20 --verbose
```

2.5 What each command reads and writes

Step	Reads	Writes
3	Inputs/Photometry/3_LCs_extrapolated/AT2017gfo.dat spectrum list (for spectroscopy Modified Julian Dates)	Outputs/AT2017gfo/fitted_phot4mangling_AT2017gfo.dat, fitted_phot_logspace_AT2017gfo.dat, optional fittedGP_*.pdf
4 (groups only)	2_spec_lists_smoothed/AT2017gfo.dat	Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json
4 (scale)	Smoothed spectra + groups JSON	Inputs/Spectroscopy/2_spec_prescaled/AT2017gfo/2_spec_lists_prescaled/AT2017gfo.list, scale report and diagnostics
5	Prescaled list + spectra, fitted_phot4mangling_*.dat, filters, raw photometry for band Modified Julian Date windows	Outputs/AT2017gfo/mangled_spectra/, AT2017gfo_band_mjd_ranges.json (if missing), figs/mangle_*
6	Mangled spectra, both fitted photometry tables, prescaled list, groups JSON	Outputs/AT2017gfo/twodim_iter/ (iterations, metrics, final products, diagnostics)

2.6 Expected artifacts checklist

Step	Key outputs to verify
3	fitted_phot4mangling_AT2017gfo.dat and fitted_phot_logspace_AT2017gfo.dat exist and are non-empty
4	Prescaled directory contains one file per smoothed spectrum; groups JSON lists XSHOOTER triplets
5	mangled_spectra/ has one mangled file per successfully constrained epoch; epoch PDFs under figs/mangle_epoch/
6	twodim_iter/final/full_gp/ has extended spectra; twodim_iter/diagnostics_summary/index.html opens; gp_vs_mangled PDFs show full instrument wavelength spans

Runtime for step 6 depends on iteration count and machine; a short debug run with `--max-iters 2` is useful before a full `--max-iters 20` production pass.

3. Input data reference

3.1 Photometry stages

Stage folder	Written by	Role
Inputs/Photometry/1_LCs_flux_raw/	User (or upstream reduction)	Observed fluxes; also used to auto-build <SN>_band_mjd_ranges.json in step 5
Inputs/Photometry/2_LCs_dust_corrected/	Notebook 1	Dust-corrected fluxes
Inputs/Photometry/3_LCs_extrapolated/	Notebook 2	Input to step 3 light-curve Gaussian Process

Stage folder	Written by	Role
Inputs/Photometry/3_LCs_early_extrapolated/	Notebook 2 (intermediate)	Early-only preview or intermediate products in some workflows

Legacy checkouts may still have 4_LCs_late_extrapolated/. The configuration module falls back to that path for the extrapolated stage if 3_LCs_extrapolated/ is missing, and emits a warning. Prefer migrating files to 3_LCs_extrapolated/.

Filename: always <SN>.dat (example: AT2017gfo.dat).

Extrapolated columns (typical): MJD, band, Flux, Flux_err, FilterSet, Instr, Phase, Log_Phase, Log_Flux, Log_Flux_err.

Raw columns (typical): MJD, Mag, Mag_err, Flux, Flux_err, band, Instr, FilterSet.

Fluxes are linear F_λ -like band fluxes in the same system used by the filter tables. Log columns use $\log_{10}$ flux.

3.2 Spectroscopy stages

Folder	Role
Inputs/Spectroscopy/1_spec_original/<SN>/	Unsmoothed spectrum files
Inputs/Spectroscopy/1_spec_lists_original/	Original list files (<SN>.list)
Inputs/Spectroscopy/2_spec_smoothed/<SN>/	Smoothed spectra (notebook 0.1 or user-provided)
Inputs/Spectroscopy/2_spec_lists_smoothed/	List pointing at smoothed files
Inputs/Spectroscopy/2_spec_prescaled/<SN>/	Flux-rescaled arms after step 4
Inputs/Spectroscopy/2_spec_lists_prescaled/	List pointing at prescaled files (used by steps 5–6 when USE_PRESCALED_SPECTRA=True)

List format: three whitespace-separated fields per line — Modified Julian Date, phase (days), path to spectrum file. Paths may be absolute; the prescale helpers rewrite stale roots that still contain older repository names when possible.

Spectrum file format: ASCII with wavelength (angstroms), flux, and flux error. A header line starting with # is allowed:

```
#wls    flux    flux_err
3725.6000  1.8021e-15  4.1786e-16
```

Naming conventions that matter for grouping:

- XSHOOTER arms: filenames containing UVB_, VIS_, NIR_ (or similar) near the same Modified Julian Date are grouped together.
- Single-instrument spectra (for example Magellan/LDSS3) remain ungrouped individuals unless you edit the groups JSON.

3.3 Filters

Throughput tables live under Inputs/Filters/GeneralFilters/ for the default AT2017gfo path. The mangling code resolves each band name to a transmission file. Events that need Carnegie Supernova Project (CSP) natural-system filters can be listed in CSP_SNE inside pipeline_config.py so that CSP-specific filter directories are preferred.

3.4 Supernova info table

Inputs/SNe_Info/info.dat holds coordinates and type fields used by the dust-correction notebook (Milky Way extinction toward the transient). Add one row per new object following the header comments in that file.

3.5 Scale groups JSON

Step 4 writes Outputs/<SN>/<SN>_spec_scale_groups.json. A minimal structure looks like:

```
{
  "default_output_mode": "scale_only",
  "groups": [
    {
      "id": "group_suggested_000",
      "members": [
        "NIR_AT2017gfo..._MJD-57983.969001....dat",
        "UVB_AT2017gfo..._MJD-57983.969002....dat",
        "VIS_AT2017gfo..._MJD-57983.969003....dat"
      ],
      "merge_order": ["uvb", "vis", "nir"],
      "output_mode": "scale_only",
      "reason": "same_time_cluster"
    }
  ],
  "ungrouped": [
    "57983.01825_LDSS3_Magellan.dat"
  ]
}
```

- **members:** basenames of spectrum files that share a flux scale.
- **merge_order:** arm order for bridging and optional merging (uvb, vis, nir).
- **output_mode:** scale_only keeps separate files (default); merge_join writes one stitched spectrum per group.
- **ungrouped:** spectra scaled only to themselves (factor 1.0) and mangled individually.

Edit this file carefully after --write-groups-only. Incorrect groups produce seam jumps and poor bundle mangling.

4. Notebooks — guidance

Each notebook should call into pipeline_config (or bootstrap_runtime) rather than hard-coding absolute paths. Set sname to your object and ensure COCO_PATH is exported before launching Jupyter.

4.1 0.1_Smooth_spectra_KN.ipynb

Purpose. Reduce noise in raw spectra while preserving broad continuum shape needed for later synthetic photometry.

Typical cell order. Import and paths → choose smoothing parameters → loop over the original list → write smoothed files and an updated list under 2_spec_smoothed/ and 2_spec_lists_smoothed/.

Outputs. Smoothed ASCII spectra and <SN>.list in the smoothed folders.

Common failures. Missing original list path; spectrum files with wrong column order; writing to a path outside COCO_PATH so later steps cannot find the list.

4.2 1_LC_DustCorrection_KN.ipynb

Purpose. Correct photometry for Milky Way (and optional host) extinction so fluxes are closer to intrinsic values before light-curve fitting.

Typical cell order. Load `info.dat` and raw light curve $\rightarrow$ compute A_λ from extinction law and $E(B - V)$ $\rightarrow$ write dust-corrected light curve.

Outputs. `Inputs/Photometry/2_LCs_dust_corrected/<SN>.dat`.

Common failures. Missing coordinates in `info.dat`; filter names in the light curve that do not match filter tables.

4.3 2_LC_modelRising_KN_fullfit_log.ipynb

Purpose. Model the early rising light curve (Bazin and related functional forms), optionally force zero flux at explosion, extrapolate late times where needed, and commit the explosion Modified Julian Date used as phase zero downstream.

Typical cell order. Load dust-corrected photometry $\rightarrow$ preview rising fits per band $\rightarrow$ commit extrapolation $\rightarrow$ write `3_LCs_extrapolated/<SN>.dat` $\rightarrow$ update or confirm `SN_EXPLOSION_MJD` in configuration.

Outputs. Extrapolated light curve under `Inputs/Photometry/3_LCs_extrapolated/`; possible intermediate early-extrapolation products.

Common failures. Explosion time inconsistent with configuration; excluding too many bands so some filters never get a rising model.

4.4 LCfit_publication_plots.ipynb (optional)

Purpose. Publication-quality light-curve figures after step 3 has written the Gaussian Process fit products.

When to run. After `3_LCfit_KN_log.py` succeeds.

4.5 Downstream QA notebooks (7.5_*.ipynb, bolometric_luminosity.ipynb, plotting-final.ipynb)

Purpose. Compare synthetic photometry from final spectra to the light-curve fit; overlay spectra; construct bolometric luminosity; assemble final figures.

Configuration. Keep `USE_ITER_GP_MANGLE_FINAL = True` (default) so path helpers resolve `twodim_iter/final/` and the QA export under `FINAL_spectra_2dim/`. Re-run the configuration and data-directory cells after changing that flag.

Prerequisites. A finished step 6 run with `final/full_gp/` populated (and QA export enabled if you rely on `_FINAL_spec` naming).

Common failures. Pointing at empty legacy directories because the iteration flag is off; looking for calendar Modified Julian Date stems when files are keyed by log-phase stems from the Gaussian Process grid.

5. Script drivers

All drivers live in `Codes/` and accept `--sname` and usually `--coco-path`. Defaults come from `pipeline_config.py`.

5.1 3_LCfit_KN_log.py

Science role. For each photometric band, fit a one-dimensional Gaussian Process to $\log_{10}$ flux versus time (or log phase). Evaluate the posterior at every spectroscopy Modified Julian Date (for mangling constraints) and on a dense log-phase grid (for the two-dimensional Gaussian Process training table).

Implementation. [helper_scripts/lc_gp_fit.py](#) with kernels from [lc_gp_kernels.py](#).

Inputs. Extrapolated light curve; spectrum list for epoch list; optional explosion anchor.

Outputs. `fitted_phot4mangling_<SN>.dat`, `fitted_phot_logspace_<SN>.dat`, optional summary PDFs.

Flag	Default	Effect
<code>--sname</code>	<code>SNNAME_DEFAULT</code>	Target object
<code>--coco-path</code>	<code>COCO_PATH</code>	Repository root
<code>--kernel-settings</code>	None	Per-band kernel JSON overrides
<code>--save-general-plots /</code>	on	Summary <code>fittedGP_*.pdf</code>
<code>--no-save-general-plots</code>		
<code>--save-per-filter-plots</code>	off	One PDF per filter under <code>lc_gp_per_filter/</code>
<code>--anchor-t0-in-lc-gp</code>	config	Include explosion-time anchor in training
<code>--append-t0-row</code>	off	Extra row in the logspace photometry file
<code>--exclude-filt</code>	None	Override band exclusion list

Worked examples.

Standard production

```
python 3_LCfit_KN_log.py --sname AT2017gfo
```

Per-filter diagnostic PDFs

```
python 3_LCfit_KN_log.py --sname AT2017gfo --save-per-filter-plots
```

5.2 4_Scale_spectra_KN.py

Science role. Multi-arm spectrographs (especially XSHOOTER) deliver separate UVB, VIS, and NIR files whose absolute flux calibrations may disagree in overlap regions. This step estimates multiplicative scale factors so arms join smoothly before mangling.

Implementation. [helper_scripts/spectra_pre_scale.py](#).

Inputs. Smoothed list and spectra; optional existing groups JSON.

Outputs. Prescaled spectra and list; groups JSON; scale report; optional HTML diagnostics under `spec_scale_diagnostics/`.

Flag	Effect
<code>--write-groups-only</code>	Write or refresh the groups JSON template; do not scale
<code>--groups-json</code>	Override groups path
<code>--output-mode</code>	<code>scale_only</code> or <code>merge_join</code>
<code>--chain-mode</code>	Sequential arm bridging along <code>merge_order</code>
<code>--gap-max-a</code>	Maximum gap (angstroms) for gap-seam scaling
<code>--write-diagnostics</code>	Seam QA under <code>spec_scale_diagnostics/</code>
<code>--same-time-minutes</code>	Clustering window for suggested groups
<code>--verbose</code>	Extra logging

Worked examples.


```
# Always do this two-step dance for a new object
python 4_Scale_spectra_KN.py --sname AT2017gfo --write-groups-only
# edit Outputs/AT2017gfo/AT2017gfo_spec_scale_groups.json
python 4_Scale_spectra_KN.py --sname AT2017gfo --write-diagnostics
```

5.3 5_Mangle_spectra_KN_log.py

Science role. At each spectroscopy epoch, compare synthetic photometry of the (prescaled) spectrum to the step-3 light-curve fit in every overlapping band. Fit a smooth mask $m(\lambda)$ in log flux so that after mangling, band fluxes match the light curve. Bundle-aware mode fits one shared mask per scale group using stitched synthetic photometry across arms.

Implementation. [helper_scripts/mangle_spectra_log.py](#).

Inputs. Prescaled list (default), `fitted_phot4mangling_*.dat`, filters, groups JSON, raw photometry for band Modified Julian Date ranges.

Outputs. `mangled_spectra/`; auto-written `*_band_mjd_ranges.json` when missing; `figs/mangle_nb5/` and optional `figs/mangle_epoch/`.

Flag	Effect
<code>--bundle-aware / --no-bundle-aware</code>	Shared mask per scale group
<code>--save-diagnostics</code>	HTML and PDF under <code>figs/mangle_nb5/</code>
<code>--save-epoch-plots</code>	Per-epoch PDFs under <code>figs/mangle_epoch/</code>
<code>--run-both-for-diag</code>	Per-arm and bundle compare (diagnostics only)
<code>--mangle-kernel-mode</code>	<code>fixed_5</code> (PyCoCo parity, default) or <code>kernel_divide_scaled</code>
<code>--kernel-divide</code>	Numerator when mode is <code>kernel_divide_scaled</code> (default 800)
<code>--groups-json</code>	Override groups path
<code>--verbose</code>	Extra logging

Worked examples.

```
# Production with epoch plots
python 5_Mangle_spectra_KN_log.py --sname AT2017gfo --save-epoch-plots
```

```
# Compare kernel modes for diagnostics
python 5_Mangle_spectra_KN_log.py --sname AT2017gfo --mangle-kernel-mode fixed_5
python 5_Mangle_spectra_KN_log.py --sname AT2017gfo --mangle-kernel-mode kernel_divide_scaled --kernel-
```

If `Outputs/<SN>/<SN>_band_mjd_ranges.json` is missing, step 5 builds it from `Inputs/Photometry/1_LCs_flux_raw/<SN>`. You do not need legacy repository-root Modified Julian Date CSV files.

5.4 6_Iterative_GP_mangle_KN.py

Science role. Outer loop: fit a two-dimensional Gaussian Process on mangled spectra plus photometry; extract the surface at observed spectroscopy epochs; remove the current mangling mask (demangle); build a new mask from demangled Gaussian Process spectra versus the light-curve fit (often on the full Gaussian Process wavelength grid); apply the new mask to **prescaled originals**; repeat until photometry closure or the iteration cap.

Implementation. [helper_scripts/iterative_gp_mangle.py](#), with Gaussian Process fitting via [iter_gp_grid.py](#) and the vendored [twodim_gp/](#) package.

Inputs. Step 5 mangled spectra (seed for `iter_00`), both fitted photometry tables, prescaled list and files, groups JSON, filters.

Outputs. `Outputs/<SN>/twodim_iter/` including per-iteration mangled spectra, predictions, figures, metrics, and `final/`.

Flag	Effect
<code>--max-iters</code>	Cap (config default 20)
<code>--phot-convergence-frac</code>	Stop when max relative photometry error is below this
<code>--bundle-aware</code>	Bundle remangle
<code>--seed-from-nb5</code>	Copy step-5 mangled spectra into <code>iter_00</code>
<code>--warm-start</code>	Warm-start Gaussian Process hyperparameters from the previous iteration
<code>--save-diagnostics</code>	Write figure suites and metrics
<code>--t0-fix</code>	Override explosion Modified Julian Date
<code>--verbose</code>	Extra logging

Worked examples.

Short debug

```
python 6_Iterative_GP_mangle_KN.py --sname AT2017gfo --max-iters 2 --verbose
```

Production

```
python 6_Iterative_GP_mangle_KN.py --sname AT2017gfo --max-iters 20 --verbose
```

Default `ITER_MANGLE_USE_GP_WAVELENGTH_GRID=True` means remangle constraints are built from a demangled Gaussian Process SED sampled across the **full training wavelength range**, not only each arm's native span. The new mask is still applied to the original prescaled wavelength grids.

5.5 `clear_sn_run.py`

Role. Delete or list output products for one supernova so you can re-run cleanly.

Flag	Effect
<code>--sname</code>	Required object name
<code>--dry-run</code>	Print paths only
(default)	Empty <code>Outputs/<SN>/</code> but keep the directory
<code>--include-input-intermediates</code>	Also delete prescaled spectroscopy directory and list (requires <code>--yes</code>)
<code>--yes</code>	Confirm destructive actions

Warning. `--include-input-intermediates` removes `Inputs/Spectroscopy/2_spec_prescaled/<SN>/` and the matching list. Re-run step 4 afterward.

6. Iteration plot layout and quality assurance

Per-iteration figures land under `Outputs/<SN>/twodim_iter/iter_KK/figs/`.

Subfolder	Content
<code>gp_surface/</code>	Heatmaps of the two-dimensional Gaussian Process mean and uncertainty, wavelength and phase slices, training residuals and coverage

Subfolder	Content
<code>gp_vs_mangled/</code>	Input mangled spectrum versus Gaussian Process extract. For individuals, the x-axis is the full prescaled wavelength span ; the Gaussian Process curve is interpolated onto that grid. Groups are stitched before plotting
<code>mangle_delta/</code>	Difference between the new mangling and the input mangling for that iteration
<code>residuals/</code>	Gaussian Process minus mangled, compared to uncertainties, on the full mangled wavelength span
<code>phot_lc/</code>	Posterior light curves and mangled synthetic photometry versus <code>gp_fit</code> targets
<code>debug/bundle_scaling/</code>	Optional bundle-pair scaler plots when <code>PLOT_BUNDLE_SCALING_QA=True</code>

Run-level metrics live in `twodim_iter/metrics/` (for example `chi2_vs_iter.png`, `lengthscales_vs_iter.png`). Gaussian Process inference caches (not plots) live under `iter_KK/gp_runs/inference/`. Open `twodim_iter/diagnostics_summary/index.html` for a clickable overview.

Detailed physical data flow for each remangle iteration is documented in [Codes/helper_scripts/docs/ITERATIVE_MANGLING_I](#)

7. pipeline_config.py reference

Edit [Codes/pipeline_config.py](#) or override via command-line flags where supported. Change a knob only when you understand which stage reads it.

7.1 Object identity and paths

Constant	Default	When to change
<code>SNNNAME_DEFAULT</code>	<code>AT2017gfo</code>	Default <code>--snnname</code> for scripts
<code>SN_EXPLOSION_MJD</code>	per-object dict	Phase zero / t_0 ; must match notebook 2
<code>SN_REDSHIFT</code>	per-object dict	Distance and rest-frame conversions where used
<code>CSP_SNE</code>	<code>()</code>	Add names that need CSP filter routing
<code>COCO_PATH</code>	env or auto	Override repository root

7.2 Prescale (step 4)

Constant	Default	When to change
<code>SPEC_SCALE_OUTPUT_MODE</code>	<code>scale_only</code>	Use <code>merge_join</code> only if you want one file per group
<code>SPEC_SCALE_CHAIN_MODE</code>	<code>False</code>	<code>True</code> for sequential bridging along
<code>SPEC_SCALE_STAR_BRIDGE_FALLBACK</code>	<code>True</code>	<code>merge_order</code>
<code>SPEC_SCALE_GAP_MAX_A</code>	400	One-hop bridge when the arm is not adjacent to the anchor
<code>SPEC_SCALE_SAME_TIME_MINUTES</code>	<code>15</code>	Maximum gap (angstroms) allowed for gap-seam scaling
		Clustering window for suggested groups

7.3 Mangling (step 5 and remangle in step 6)

Constant	Default	When to change
USE_PRESCALED_SPECTRA	True	Leave on for production; off only for legacy smoothed-only tests
MANGLE_BUNDLE_AWARE	True	Shared mask per group (recommended for XSHOOTER)
MANGLE_GP_KERNEL_MODE	fixed_5	PyCoCo parity; switch to <code>kernel_divide_scaled</code> only for experiments
MANGLE_GP_KERNEL_FIXED	5.0	Fixed Matérn 3/2 length in normalized log-wavelength when mode is <code>fixed_5</code>
MANGLE_KERNEL_DIVIDE	800	Used only in <code>kernel_divide_scaled</code> mode
MANGLE_BUNDLE_STITCH_SYNPHOT	True	Integrate filters on stitched multi-arm spectra
MANGLE_RUN_BOTH_FOR_DIAG	False	Turn on to compare per-arm versus bundle once

7.4 Iterative two-dimensional Gaussian Process (step 6)

Constant	Default	When to change
ITER_GP_MANGLE_MAX_ITERS	20	Lower for debugging; raise only if convergence is slow but improving
PHOT_CONVERGENCE_FRAC	0.05	Primary stop: max relative synthetic photometry error
MASK_CONVERGENCE_EPS	1e-4	Secondary logging on mask change
ITER_MANGLE_USE_GP_WAVELENGTH_GRID	True	Full-wavelength remangle constraints (recommended)
ITER_GP_SEED_FROM_NB5	True	Seed <code>iter_00</code> from step 5 mangled spectra
ITER_GP_WARM_START	True	Reuse hyperparameters between iterations
ITER_GP_EXPORT_FULL_GP	True	Write extended spectra under <code>full_gp/</code>
ITER_EXPORT_FINAL_SPEC_FOR_QA	True	Copy QA-named spectra for notebooks 7.5
USE_ITER_GP_MANGLE_FINAL	True	Notebooks 7.5 read iteration final products
ITER_GP_DIAG_FIGS	True	Write Gaussian Process surface diagnostic figures
PLOT_BUNDLE_SCALING_QA	False	Extra debug figures for bundle scaling

Two-dimensional inference hyperparameters (kernel family, additive scales, jitter floors, early-time constraints) live in `GP_INFERENCE_KWARGS`. Treat those as an advanced surface; defaults match the collaborator production configuration described in the writeup.

7.5 Plotting style

Band colors, markers, and exclusion lists are loaded from [Codes/filter_plot_config.json](#). Edit that JSON when a band should be omitted from fits or drawn in a specific color.

Full constant list: see the source file and its grouped sections.

8. Troubleshooting

Symptom	Likely cause	What to do
<code>no_phot_constraints</code> in step 5	No light-curve-fit bands overlap the spectrum epoch within band Modified Julian Date windows	Check <code>*_band_mjd_ranges.json</code> , band names, and whether step 3 wrote finite fits for those bands
Missing <code>*_band_mjd_ranges.json</code>	Outputs were cleared and step 5 has not run yet	Re-run step 5; it regenerates the JSON from <code>1_LCs_flux_raw</code>
Empty or short prescaled list	Step 4 not run, or smoothed list empty	Re-run notebook 0.1 and step 4; verify list paths resolve under <code>COCO_PATH</code>
Wild oscillations in mangling mask	Kernel mode not <code>fixed_5</code> , or too few constraints	Prefer <code>--mangle-kernel-mode fixed_5</code> ; inspect epoch plots and used filters
<code>n_skipped_extract</code> in step 6	Epoch was not mangled in step 5	Expected for epochs without photometry constraints; fix step 5 if the epoch should have been mangled
Empty bundle epoch plot	No photometry constraints on the reference arm	Inspect bundle diagnostics; confirm groups and filter overlap
Missing <code>figs/gp_surface</code>	Gaussian Process fit failed or <code>ITER_GP_DIAG_FIGS=False</code>	Check iteration logs; enable diagnostics; verify <code>predictions.npz</code> exists
<code>gp_vs_mangled</code> shows only ~ 200 Å	Outdated plotting bug (fixed): truncating prescaled arrays to Gaussian Process grid length	Update <code>iter_plot_suite.py</code> and regenerate figures
Iteration never converges	Photometry targets inconsistent, or mask unstable	Inspect <code>phot_lc/</code> and <code>metrics/</code> ; try warm-start on; verify step 3 and step 5 quality first
Notebooks 7.5 find no spectra	<code>USE_ITER_GP_MANGLE_FINAL</code> false or <code>final/full_gp/</code> empty	Set the flag true; confirm step 6 export flags; re-run config cells

Run the unit tests after significant code or environment changes:

`cd Codes`

`PYTHONPATH=./helper_scripts python -m unittest discover -s helper_scripts/tests -v`

9. Developer documentation and PDF generation

Internal phase notes (still useful for contributors):

- [Prescale](#)
- [Iterative Gaussian Process + mangling](#)
- [Physical data flow](#)
- [Downstream QA](#)
- [Deprecated modules](#)

Scientific equations for the whole pipeline: [PIPELINE_WRITEUP.md](#).

Build PDFs (requires [pandoc](#); the writeup also needs a TeX distribution providing `pdflatex`):

`docs/build_user_guide_pdf.sh`

`docs/build_writeup_pdf.sh`